

Lab 3 – Challenge: Airport Departure Management System

Scenario

You have been asked to create a console-based system for managing departures at a small airport.

The airport has several scheduled flights. During the day, your program must process flights, classify their status, display departure information, search for flights and allow a user to interact with the system through a menu.

Unlike the standard lab, this challenge does not tell you exactly where to use each type of loop or condition.

You are expected to decide when and how to use conditions, for loops, while loops, range, enumerate, break and continue.

Your solution should build on the Python collections covered in the previous lesson.

Part 1 – Create the departure data

Create a collection containing at least 10 flights.

Each flight should contain information such as:

- flight number
- destination
- departure time
- gate
- passengers
- maximum capacity
- delay in minutes
- cancelled status

For example, your data should make it possible to represent a flight that:

- departs for London
- leaves at 14:30
- uses Gate B4
- has 132 passengers
- has capacity for 180 passengers
- is delayed by 25 minutes
- is not cancelled

You decide how the complete data structure should be designed.

Part 2 – Process the departure board

Create a departure board that processes all flights in your collection.

For every flight, print useful information such as:

```
SK142 - London - 14:30 - Gate B4
```

Number the flights when displaying them so that the output looks something like:

```
1. SK142 - London - 14:30 - Gate B4
2. LH231 - Berlin - 15:00 - Gate A2
3. BA442 - Manchester - 15:20 - Gate C1
```

Do not manually write a separate `print()` statement for every flight.

Part 3 – Determine flight status

The departure board should automatically determine the status of each flight.

Use the following rules:

- Cancelled → "CANCELLED"
- Delay of 60 minutes or more → "SEVERELY DELAYED"
- Delay of 20–59 minutes → "DELAYED"
- Delay of 1–19 minutes → "SLIGHT DELAY"
- No delay → "ON TIME"

A cancelled flight should always display "CANCELLED" regardless of its delay.

Example:

```
SK142 - London - DELAYED
LH231 - Berlin - ON TIME
BA442 - Manchester - CANCELLED
```

Think carefully about the order of your conditions.

Part 4 – Analyse the flights

Use your flight data to calculate and display:

- total number of scheduled flights
- number of cancelled flights
- number of delayed flights
- number of flights departing on time
- total number of passengers
- average number of passengers per flight
- flight with the largest number of passengers
- number of flights with more than 80% of their capacity filled

For this part, do not use `max()` to find the flight with the largest number of passengers.

Your program should determine it while processing the data.

Part 5 – Search for a flight

Allow the user to enter a flight number.

Search through the flight data for that flight.

If the flight exists, display its information.

For example:

```
Enter flight number: SK142

Destination: London
Departure: 14:30
Gate: B4
Passengers: 132
Status: DELAYED
```

Once the flight has been found, there is no reason to continue searching through the remaining flights.

If no matching flight exists, display:

```
Flight not found.
```

Think carefully about how your program can determine whether the flight was actually found.

Part 6 – Handle unavailable flights

Add some special cases to your flight data.

At least:

- one flight should be cancelled
- one flight should have no assigned gate
- one flight should have 0 passengers

When displaying the departure board:

- cancelled flights should still be displayed
- flights without a gate should display "Gate not assigned"
- flights with 0 passengers should not be included when calculating the average passenger count for active flights

Use conditions to handle these situations rather than creating separate versions of the data.

Part 7 – Gate overview

The airport wants a simple overview of its gates.

Create at least:

- 3 terminal letters: A, B, C
- 4 gate numbers per terminal

Generate and print all possible gate combinations:

```
Gate A1
Gate A2
Gate A3
Gate A4
Gate B1
Gate B2
...
Gate C4
```

Do not manually write all twelve combinations.

Part 8 – Interactive Airport Menu

Create a menu that repeatedly allows the user to choose an action.

For example:

```
AIRPORT DEPARTURE SYSTEM

1. View all flights
2. View delayed flights
3. View cancelled flights
4. Search for a flight
5. View flight statistics
6. Quit

Choose an option:
```

The menu should continue running until the user chooses to quit.

Option 1 – View all flights

Display all scheduled flights.

Option 2 – View delayed flights

Display only flights that have a delay.

Cancelled flights should not be included in this view.

Option 3 – View cancelled flights

Display only cancelled flights.

Option 4 – Search for a flight

Ask the user for a flight number and display the matching flight.

If the flight does not exist, display an appropriate message.

Option 5 – View flight statistics

Display the statistics you calculated earlier.

Option 6 – Quit

End the program and display a short goodbye message.

If the user enters an invalid menu option, the program should display an error message and show the menu again.

Part 9 – Control the processing

Review your program and identify situations where continuing a loop is unnecessary.

Use break and/or continue where they genuinely improve the program.

For example:

- stop searching after finding the requested flight
- skip data that should not be processed in a particular calculation
- exit the menu when the user chooses to quit

Do not use break or continue simply to demonstrate that you know the syntax. They should have a clear purpose.

Final Challenge – Airport Operations Report

Extend your program so that it produces a summary such as:

```
AIRPORT OPERATIONS REPORT

Scheduled flights: 10
Cancelled flights: 2
Delayed flights: 4
On-time flights: 4

Passengers today: 1247

Busiest flight:
SK142 - London - 176 passengers

Flights above 80% capacity:
SK142 - London
LH231 - Berlin
BA221 - Manchester
```

Then add one additional analysis of your own.

For example:

- destination with the most passengers
- average delay
- longest delay

- busiest gate
- percentage of cancelled flights

You decide how to calculate and present it.

Design Challenge

Before considering the lab complete, look through your solution and identify at least two places where your first solution could be improved.

For each one, write a short comment explaining:

1. What your original solution did.
2. What you changed.
3. Why the new solution is better.

The improvement could relate to:

- unnecessary repetition
- condition order
- loop choice
- readability
- unnecessary processing
- data structure
- variable naming